Universidad Distrital Francisco José de Caldas

Computer Engineering Program

Computer Sciences I — 2026-I

Plague Containment

Technical Report — Checkpoint Week 8

Team 16

Nelson David Molina Ramos	`Data Structures (C++)`
Brayan Estiven Aguirre Aristizabal	`Algorithms (Python)`
Juan Sneyder Méndez Gil	`Interface & Integration`

Bogotá, Colombia

March 27, 2026

Contents

1 Abstract **3**

2 Introduction **4**

3 Literature Review **4**

4 Objectives **5**

5 Scope **5**

6 Assumptions **6**

7 Limitations **6**

8 Methodology **7**

8.1 City Grid Model 7

8.2 Greedy Vaccination Algorithm 8

8.3 Backtracking Quarantine Algorithm 8

8.4 Game Turn Structure 10

8.5 System Architecture 10

8.6 JSON Schema Design 11

8.7 Interface Design 13

9 Implementation and Results **13**

10 Discussion **14**

11 Conclusions **16**

Glossary **18**

List of Figures

1 Example of the game grid mid-game. Green cells are vaccinated, red cells are infected, blue cells are quarantined, and yellow-green cells are healthy. 7
2 Sequence of events inside one game turn. 10
3 Data flow between the three system components through JSON files. Numbers indicate the order of operations within one turn. 11
4 Basic wireframe of the Pygame interface showing the grid, info panel, and action buttons. 13
5 Detailed wireframe of the game interface. Each cell shows its risk level. Colors encode district states. The yellow-bordered cell marks the greedy suggestion. 14

List of Tables

1 District states and their meaning in the game. 7
2 Responsibility of each system component. 11
3 Design deliverables per team member at Week 8 checkpoint. 15

1. Abstract

Plague Containment is a turn-based strategy game designed as part of the Computer Sciences I course project at Universidad Distrital Francisco José de Caldas. The game models a city as an 8×8 grid of districts, each carrying an infection risk level, and challenges the player to stop a spreading plague using two algorithmic tools: a greedy vaccination advisor that always targets the highest-risk unprotected district on the infection border, and a backtracking quarantine planner that searches for a ring of districts enclosing the infected zone without isolating any healthy area. This report presents the complete design of the system, including the city model, the pseudo-code of both algorithms, the three-layer architecture (C++ engine, Python algorithms, Pygame interface), the JSON communication schema that bridges all layers, and the wireframe of the game interface. No implementation has been completed at this stage; the next steps are to code the algorithms, build the interface, and develop the C++ engine.

2. Introduction

Epidemic spread is a well-studied problem both in epidemiology and in computer science. Models that represent populations as graphs or grids have been used to simulate the progression of infections and to evaluate containment strategies [3]. Algorithmic approaches such as greedy heuristics and exhaustive search with backtracking are fundamental tools for solving optimization and constraint-satisfaction problems of this kind, as described in standard references on algorithms [1] and artificial intelligence [2].

Plague Containment applies these ideas in the context of a video game. The player is placed in charge of a city represented as an 8×8 grid of districts. Each district carries an infection risk level, and the plague spreads every turn to neighboring districts. The player has two tools at their disposal: they may vaccinate a district to make it permanently immune, or they may trigger a quarantine perimeter that isolates the infected zone. Two algorithms assist the player: a greedy advisor that recommends the single most dangerous unprotected neighbor of the infected zone, and a backtracking planner that automatically finds a valid quarantine ring while guaranteeing that no healthy district becomes disconnected from the rest of the city.

The goal of this report is to document the full design of the game for the Week 8 checkpoint. It covers the city model, the algorithmic design and pseudo-code, the system architecture, the JSON communication schema, and the interface wireframe. Implementation of the system is planned as the next phase of the project.

3. Literature Review

The design of Plague Containment draws on three established areas of computer science and mathematics.

Epidemic modeling on networks and grids has a long history in applied mathematics. Pastor-Satorras et al. [3] provide a comprehensive review of epidemic processes on complex networks, showing how the local connectivity of a node directly influences how quickly an infection spreads to its neighbors. This finding motivates the risk-level model used in Plague Containment: districts with higher risk values represent nodes that are more likely to propagate the infection, making them the priority targets for the greedy vaccination strategy.

Greedy algorithms are covered extensively in Cormen et al. [1], which establishes that a greedy approach is appropriate when a locally optimal choice at each step leads to a globally acceptable result. In the vaccination problem, the immediate danger at each turn comes from the unprotected district with the highest risk on the infection border; targeting that district first reduces the most urgent threat and is a natural application of the greedy principle.

Backtracking search is presented by both Cormen et al. [1] and Russell and Norvig [2] as the standard technique for constraint-satisfaction problems in which a partial solution must be extended step by step, with the ability to undo choices that violate constraints. The quarantine problem requires simultaneous satisfaction of two constraints — full enclosure of the infected zone and preservation of the connectivity of the healthy city — which cannot be guaranteed by a greedy selection, making backtracking the appropriate algorithmic choice.

4. Objectives

The main goal of this project is to design and implement a playable, turn-based strategy game that demonstrates the practical application of greedy and backtracking algorithms in the context of epidemic containment.

The specific objectives for the Week 8 checkpoint are:

1. Define a formal model of the city grid, including district states, risk levels, and plague spread rules.
2. Design and express in pseudo-code a greedy algorithm that recommends the best district to vaccinate each turn.
3. Design and express in pseudo-code a backtracking algorithm that finds a valid quarantine perimeter around the infected zone while preserving city connectivity.
4. Define a three-layer system architecture and a JSON communication schema that allows a C++ engine, a Python algorithm module, and a Pygame interface to exchange data without direct language bindings.
5. Produce a wireframe of the game interface that reflects the information provided by both the engine and the algorithms.

5. Scope

This report covers the design phase of the Plague Containment project (Week 8 checkpoint). It includes the city model, algorithmic design, system architecture, JSON schema, and interface wireframe. It does not include working source code, test results, or performance measurements, as those belong to subsequent checkpoints. The game is intended to run locally on a desktop machine and is designed for a single player.

The grid is fixed at 8×8 districts. Risk levels are integers from 1 to 10 assigned once at game start and do not change. The game ends when either all infected districts are contained (player wins) or the plague spreads to more than a predefined threshold of districts (player loses).

6. Assumptions

The following assumptions were made during the design of the system:

1. Adjacency is defined as sharing a side (4-connectivity: up, down, left, right). Diagonal neighbors are not considered adjacent.
2. The plague spreads with a non-zero probability to every unprotected healthy neighbor of every infected district at the end of each turn. The exact probability is a design parameter to be set during implementation.
3. A district in the *quarantined* or *vaccinated* state completely blocks plague spread; no probabilistic transmission occurs across those borders.
4. The backtracking algorithm operates on the set of healthy districts that border the infected zone. It is assumed that this set is finite and small enough for exhaustive search within the response time of a single game turn.
5. JSON file I/O is fast enough relative to the game's turn duration that it does not create noticeable latency for the player.
6. All three system components (C++ engine, Python algorithms, Pygame interface) run on the same local machine and share access to the same filesystem directory.

7. Limitations

The current design has several known limitations:

1. The backtracking quarantine algorithm explores all possible subsets of border districts. In the worst case its running time is exponential in the number of border districts. For a densely infected grid the algorithm could be slow, but in practice the border set is small and the constraint checks prune most branches early.
2. The JSON bridge introduces a file I/O overhead on every turn. For a turn-based game this overhead is acceptable, but the design is not suitable for real-time applications.
3. The current design does not include a multiplayer mode, difficulty scaling beyond risk level assignment, or an AI opponent.
4. The wireframe presented in this report is a design prototype. The final visual style, color palette, and layout may change during implementation.

5. No formal risk analysis has been performed on the probability model for plague spread. The spread mechanics described here are conceptual and will require calibration during testing.

8. Methodology

8.1 City Grid Model

The city is represented as an 8×8 grid of districts. Each cell in the grid corresponds to one district, and two districts are considered neighbors if they share a side (up, down, left, or right). Every district is always in exactly one of the four states described in Table 1.

Table 1: District states and their meaning in the game.

State	**Description**
Healthy	Not yet affected by the plague. Can become infected.
Infected	Currently carrying the plague; attempts to spread to healthy neighbors each turn.
Vaccinated	Permanently immunized by the player. Cannot become infected and blocks spread.
Quarantined	Placed under lockdown by the backtracking planner to form the containment perimeter. Blocks spread.

Each district also carries a *risk level*, which is a whole number between 1 and 10 assigned at the start of the game. A higher risk level indicates greater vulnerability to infection, for example due to high population density or poor sanitation. The risk level does not change during the game. Figure 1 shows an example state of the grid mid-game.

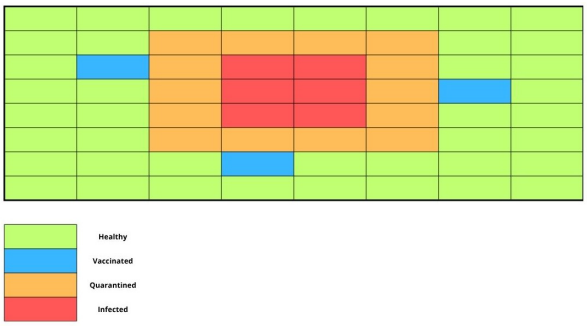

Figure 1: Example of the game grid mid-game. Green cells are vaccinated, red cells are infected, blue cells are quarantined, and yellow-green cells are healthy.

At the end of each turn the plague attempts to spread: for every infected district, each of its healthy neighbors has a chance of becoming infected. Districts in the vaccinated or quarantined state block spread entirely.

8.2 Greedy Vaccination Algorithm

The greedy algorithm helps the player decide which district to vaccinate each turn. It scans every healthy district that directly borders an infected one and recommends the district with the highest risk level. The rationale is straightforward: the most dangerous unprotected neighbor on the infection border is the one most likely to spread the plague further if left unprotected, so targeting it first provides the best immediate reduction of risk.

Greedy algorithms are appropriate when a locally optimal decision at each step leads to a globally acceptable outcome. In the vaccination problem the immediate threat is always concentrated in the border district with the highest risk, so a local scan of the border is sufficient to make a useful recommendation without looking ahead across multiple turns. Algorithm 1 shows the complete pseudo-code.

Algorithm 1 Greedy Vaccination

Input: The game grid with all district states and risk levels
Output: The best district to vaccinate this turn, or NONE

```
candidates ← empty list
for each infected district d on the grid do
    for each neighbor n of d do
        if n is HEALTHY then
            add n to candidates (avoid duplicates)
        end if
    end for
end for
if candidates is empty then
    return NONE
end if
best ← first district in candidates
for each district c in candidates do
    if c.risk > best.risk then
        best ← c
    end if
end for
mark best as VACCINATED
return best
```

8.3 Backtracking Quarantine Algorithm

The backtracking algorithm finds a set of healthy districts to quarantine so that the infected zone is completely enclosed. The solution must satisfy two conditions simultaneously:

1. **Full enclosure.** Every infected district must be completely surrounded; none of its neighbors may remain a free healthy district.

2. **No isolation.** After quarantine districts are placed, every remaining healthy district must still be reachable from the rest of the healthy city. No healthy area may become a disconnected island.

A greedy approach is not suitable here because a locally reasonable choice — for example, quarantining a district that appears necessary to close the perimeter — can accidentally cut a healthy neighborhood off from the rest of the city. Backtracking allows the algorithm to try a quarantine placement, verify both conditions, and undo the placement if either condition is violated, then explore the alternative of skipping that district, until a valid complete solution is found or all possibilities are exhausted.

Algorithm 2 is the entry point, and Algorithm 3 is the recursive function it calls.

Algorithm 2 Backtracking Quarantine — Entry Point

Input: The game grid with all district states
Output: A valid quarantine set, or NONE if impossible

```
border ← all healthy neighbors of infected districts
quarantine ← empty set
success ← SEARCH(border, index 0, quarantine)
if success then
    return quarantine
else
    return NONE
end if
```

Algorithm 3 Search — Recursive Backtracking Step

Input: Border district list, current index i, partial quarantine set *quarantine*
Output: TRUE if a complete valid quarantine was found, FALSE otherwise

```
if infected zone is fully enclosed by quarantine then
    return TRUE
end if
if no more border districts to try then
    return FALSE
end if
d ← border[i] {— Option A: quarantine district d —}
add d to quarantine
temporarily mark d as QUARANTINED on the grid
if remaining healthy districts are still all connected then
    if SEARCH(border, i + 1, quarantine) then
        return TRUE
    end if
end if{— Backtrack: undo quarantine of d —}
remove d from quarantine
restore d to HEALTHY on the grid {— Option B: skip district d —}
if SEARCH(border, i + 1, quarantine) then
    return TRUE
end if
return FALSE
```

8.4 Game Turn Structure

The greedy and backtracking algorithms serve different roles within a single turn. Figure 2 shows the sequence of events. The greedy algorithm runs at the start of the turn, before the player acts, so that the recommendation is based on the current grid state. The backtracking planner runs after the plague spreads, so that the quarantine perimeter it computes always reflects the most up-to-date infection state.

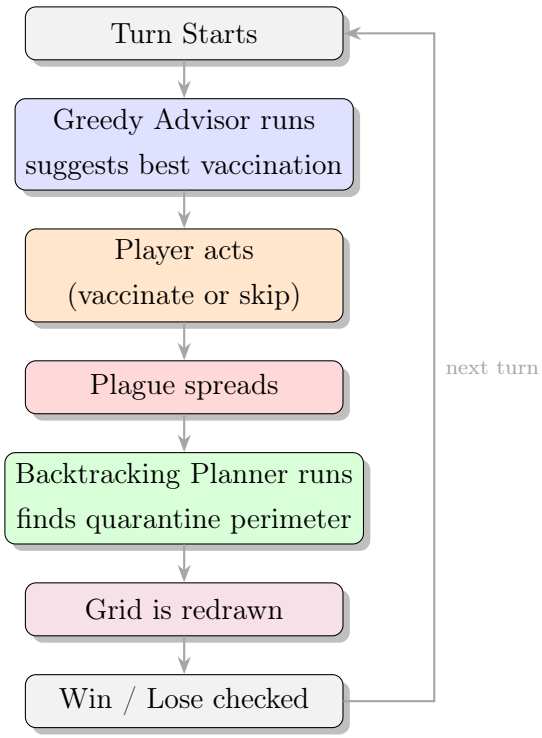

Figure 2: Sequence of events inside one game turn.

8.5 System Architecture

Plague Containment is organized in three independent layers, each implemented in a different language. The C++ engine manages the core game state, including a linked list that tracks the infection chain in insertion order and an AVL tree that indexes districts by risk level for fast lookup. The Python module implements both algorithms. The Pygame interface renders the grid, displays the algorithm suggestions, and handles player input. Table 2 summarizes the responsibility of each layer.

The three layers do not call each other directly. Instead they communicate through three shared JSON files that act as a language-neutral bridge. This design avoids the complexity of cross-language bindings and makes debugging straightforward, since any team member can open the JSON files at any point and inspect the complete game state.

Figure 3 shows the data flow among the three layers during a single turn.

Table 2: Responsibility of each system component.

Layer	Language	Responsibilities
Engine	C++	Maintain the infection chain (linked list) and district risk index (AVL tree). Update plague spread each turn. Write the updated game state to `state.json`.
Algorithms	Python	Read the game state from `state.json`. Run the greedy advisor and the backtracking planner. Write results to `result.json`.
Interface	Python/Pygame	Read `state.json` and `result.json`. Render the grid, highlights, and HUD. Capture player input and write the chosen action to `action.json`.

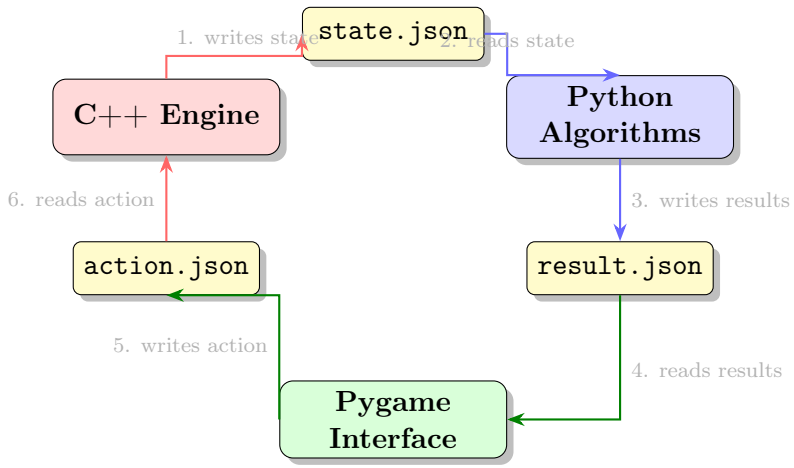

Figure 3: Data flow between the three system components through JSON files. Numbers indicate the order of operations within one turn.

8.6 JSON Schema Design

Three JSON files mediate all inter-component communication. Each file has a clearly defined structure and a single writer and one or more readers.

`state.json` is written by the C++ engine at the start of each turn. It contains the complete snapshot of the grid, including the state and risk level of every district, the infection chain in insertion order (mirroring the C++ linked list), and precomputed counters for the interface HUD. Listing 1 shows an example.

Listing 1: Example `state.json` at turn 3.

```
{
  "turn": 3,
  "grid_size": 8,
  "districts": [
    {"row": 0,"col": 0,"risk": 5,"state": "healthy" },
    {"row": 0,"col": 1,"risk": 8,"state": "infected" },
    {"row": 1,"col": 0,"risk": 3,"state": "vaccinated" },
```

```
    {"row": 1,"col": 1,"risk": 7,"state": "quarantined" }
  ],
  "infection_chain": [
    {"row": 3,"col": 4,"turn_infected": 1},
    {"row": 3,"col": 5,"turn_infected": 1},
    {"row": 2,"col": 4,"turn_infected": 2},
    {"row": 4,"col": 4,"turn_infected": 3}
  ],
  "stats": {
    "total_infected": 4,
    "total_vaccinated": 6,
    "total_quarantined": 2,
    "total_healthy": 52
  }
}
```

`result.json` is written by the Python algorithm module after processing the game state. It contains the greedy vaccination recommendation and the backtracking quarantine plan. If no valid quarantine exists, the `found` field is set to `false` and the `districts` array is empty. Listing 2 shows an example.

Listing 2: Example `result.json`.

```
{
  "greedy_recommendation": {
    "row": 2,"col": 5,
    "risk": 9,
    "reason": "highest-risk healthy neighbor"
  },
  "quarantine_plan": {
    "found": true,
    "districts": [
      {"row": 2,"col": 3},
      {"row": 4,"col": 5},
      {"row": 5,"col": 4},
      {"row": 3,"col": 6}
    ],
    "cost": 4
  }
}
```

`action.json` is written by the Pygame interface when the player makes a decision. The `action_type` field can be `"vaccinate"`, `"quarantine"`, or `"skip"`. Listing 3 shows

an example.

Listing 3: Example `action.json`.

```
{
  "turn": 3,
  "action_type": "vaccinate",
  "target": {"row": 2,"col": 5}
}
```

8.7 Interface Design

The game window is divided into two regions: the main 8×8 grid on the left and an information panel on the right. Each cell displays its risk level as a number, and distinct colors encode the four district states: white for healthy, red for infected, green for vaccinated, and purple for quarantined. The district currently recommended by the greedy advisor is marked with a yellow border. The side panel shows turn number, district counts, the greedy suggestion, the quarantine plan status, and three action buttons (VACCINATE, QUARANTINE, SKIP TURN). Figures 4 and 5 show the planned layout.

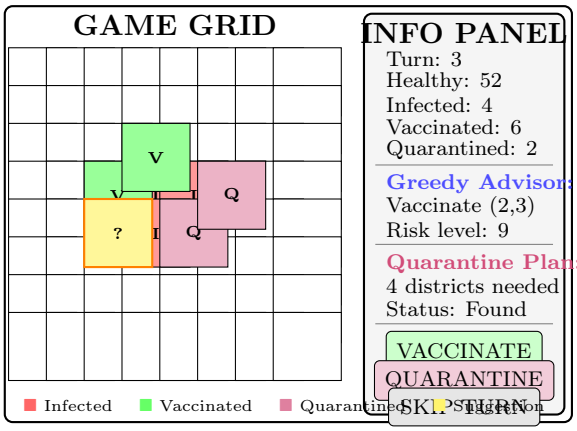

Figure 4: Basic wireframe of the Pygame interface showing the grid, info panel, and action buttons.

9. Implementation and Results

At this checkpoint the project is at the end of the design phase. No running code has been produced yet; the results of this phase are the design artifacts described in the Methodology section. This section summarizes what has been delivered and outlines the work remaining.

Table 3 lists the design deliverables produced by each team member for the Week 8 checkpoint.

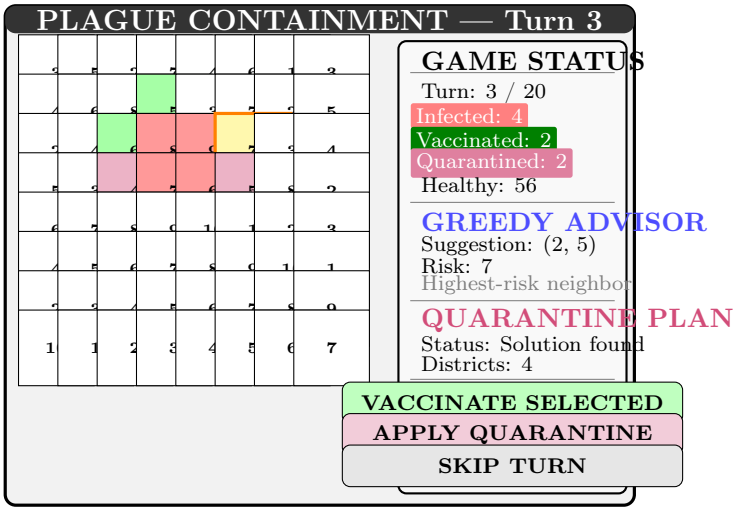

Figure 5: Detailed wireframe of the game interface. Each cell shows its risk level. Colors encode district states. The yellow-bordered cell marks the greedy suggestion.

All three design components are mutually consistent: the JSON fields in `state.json`, `result.json`, and `action.json` map directly to the data structures defined by the engine developer; the algorithm outputs match the fields read by the interface; and the wireframe reflects the information provided by both the engine and the algorithms.

The next implementation steps are:

1. Implement the infection linked list and district AVL tree in C++, and write the game loop that manages plague spread and writes `state.json`.
2. Implement the greedy and backtracking algorithms in Python, reading from `state.json` and writing to `result.json`.
3. Build the Pygame interface that renders the grid, displays algorithm suggestions, captures player input, and writes `action.json`.
4. Integrate the three layers and perform end-to-end testing.

10. Discussion

The design choices made in this phase reflect the constraints of the course and the nature of the algorithmic problems being addressed.

The decision to use a JSON file bridge rather than direct language bindings was driven by simplicity and debuggability. While tools such as `pybind11` would allow C++ and Python to call each other directly, they introduce compilation complexity that is beyond the scope of this course. The JSON approach makes every component independently testable: the C++ engine can be tested by checking the contents of `state.json`, the Python algorithms can be tested by feeding them a hand-crafted `state.json`, and the

Table 3: Design deliverables per team member at Week 8 checkpoint.

Team Member	**Role**	**Deliverables**
Nelson David Molina Ramos	Data Structures (C++)	Design and diagrams of the infection linked list and the district-risk AVL tree. Explanation of how each structure supports the game engine.
Brayan Estiven Aguirre Aristizabal	Algorithms (Python)	Pseudo-code and step-by-step explanation of the Greedy Vaccination algorithm (Algorithm 1) and the Backtracking Quarantine algorithm (Algorithms 2–3).
Juan Sneyder Méndez Gil	Interface & Integration	Pygame wireframe design (Figures 4–5), JSON schema specification (Listings 1–3), and system architecture diagram (Figure 3).

Pygame interface can be tested against a hand-crafted `result.json` without running the other layers.

The choice of a greedy algorithm for vaccination and backtracking for quarantine reflects the fundamental difference between the two problems. Vaccination is a one-shot, per-district recommendation: there is no need to plan several moves ahead, because the risk of each border district is fixed and visible. The greedy approach is therefore both appropriate and efficient. Quarantine, by contrast, is a constraint-satisfaction problem in which two conditions must hold simultaneously across the full perimeter. A greedy selection would not detect connectivity violations until after the choice is made, requiring some form of undoing anyway. Backtracking makes the search strategy explicit and guarantees that any solution found satisfies both constraints.

One area of concern for the implementation phase is the worst-case performance of the backtracking algorithm. If the infected zone grows large and spreads across the center of the grid, the border set could contain on the order of twenty or more districts, making exhaustive search potentially slow. Practical mitigation strategies include early termination when a connectivity violation is detected (already included in the pseudo-code at Algorithm 3, line 10), and ordering the border list by proximity to the infection center to find solutions faster on average.

11. Conclusions

This report has presented the complete design of Plague Containment, a turn-based strategy game that uses greedy and backtracking algorithms to assist the player in containing a spreading plague across an 8×8 city grid. The main outcomes of this design phase are a formal city model with four district states and integer risk levels, pseudo-code for both algorithms (greedy vaccination and backtracking quarantine), a three-layer system architecture connected by a JSON file bridge, a fully specified JSON schema, and two wireframes of the game interface.

The design is internally consistent: data flows unambiguously from the C++ engine to the Python algorithms to the Pygame interface and back, all without direct language bindings. The algorithmic choices are justified by the nature of each sub-problem: greedy selection is appropriate for the single-step vaccination recommendation, while backtracking is necessary for the global constraint satisfaction required by the quarantine perimeter.

Future work includes the full implementation of all three layers, automated unit testing of both algorithms, integration testing of the JSON bridge, and calibration of the plague spread probability to produce a challenging but fair gameplay experience. Potential improvements beyond the current scope include adjustable difficulty, a save/load mechanism, and performance optimization of the backtracking search for large infection zones.

11. References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[3] R. Pastor-Satorras, C. Castellano, P. Van Mieghem, and A. Vespignani, "Epidemic processes in complex networks," *Reviews of Modern Physics*, vol. 87, no. 3, pp. 925–979, 2015.

11. Glossary

AVL tree A self-balancing binary search tree where the heights of the two child subtrees of any node differ by at most one. Used in this project to index districts by risk level for fast lookup.

Backtracking An algorithmic technique that builds a solution incrementally, abandoning (backtracking) a partial solution as soon as it determines that the partial solution cannot lead to a valid complete solution.

District One cell of the 8×8 city grid. Each district has a state and a risk level.

Greedy algorithm An algorithm that makes the locally optimal choice at each step, without looking ahead, in the hope of finding a globally good solution.

Infection chain The ordered sequence of districts that have become infected, stored as a linked list in the C++ engine and serialized as the `infection_chain` array in `state.json`.

JSON (JavaScript Object Notation) A lightweight, human-readable data-interchange format used in this project as a communication bridge between the C++ engine, Python algorithms, and Pygame interface.

Linked list A data structure in which each element (node) contains a value and a pointer to the next node. Used in this project to maintain the infection chain in insertion order.

Plague spread The mechanism by which infected districts probabilistically infect their healthy neighbors at the end of each turn. Blocked by vaccinated and quarantined districts.

Pygame A Python library for writing video games, used in this project to render the graphical interface and handle player input.

Quarantine perimeter A ring of districts placed in the quarantined state that completely surrounds the infected zone without isolating any healthy district.

Risk level An integer from 1 to 10 assigned to each district at game start, representing the district's vulnerability to infection. Higher values indicate greater risk.